This works equally well for vector quantities $\mathbf{a}_i$. So, for a vertex skinned to N joints with indices j_0 through j_{N-1} and weights w_0 through w_{N-1}, we can extend Equation (12.4) as follows:

$$\mathbf{v}_{\mathrm{M}}^{\mathrm{C}} = \sum_{i=0}^{N-1} w_i \mathbf{v}_{\mathrm{M}}^{\mathrm{B}} \mathbf{K}_{j_i},$$

where $\mathbf{K}_{j_i}$ is the skinning matrix for the joint j_i.

12.6 Animation Blending

The term *animation blending* refers to any technique that allows more than one animation clip to contribute to the final pose of the character. To be more precise, blending combines two or more *input poses* to produce an *output pose* for the skeleton.

Blending usually combines two or more poses at a single point in time, and generates an output at that same moment in time. In this context, blending is used to combine two or more animations into a host of new animations, without having to create them manually. For example, by blending an injured walk animation with an uninjured walk, we can generate various intermediate levels of apparent injury for our character while he is walking. As another example, we can blend between an animation in which the character is aiming to the left and one in which he's aiming to the right, in order to make the character aim along any desired angle between the two extremes. Blending can be used to interpolate between extreme facial expressions, body stances, locomotion modes and so on.

Blending can also be used to find an intermediate pose between two known poses at *different* points in time. This is used when we want to find the pose of a character at a point in time that does not correspond exactly to one of the sampled frames available in the animation data. We can also use temporal animation blending to smoothly transition from one animation to another, by gradually blending from the source animation to the destination over a short period of time.

12.6.1 LERP Blending

Given a skeleton with N joints, and two skeletal poses $\mathbf{P}_A^{\mathrm{skel}} = \left\{ (\mathbf{P}_A)_j \right\} |_{j=0}^{N-1}$ and $\mathbf{P}_B^{\mathrm{skel}} = \left\{ (\mathbf{P}_B)_j \right\} |_{j=0}^{N-1}$, we wish to find an intermediate pose $\mathbf{P}_{\mathrm{LERP}}^{\mathrm{skel}}$ between these two extremes. This can be done by performing a *linear interpolation* (LERP) between the local poses of each individual joint in each of the two

source poses. This can be written as follows:

$$\begin{aligned}(\mathbf{P}_{\text{LERP}})_j &= \text{LERP}\left((\mathbf{P}_A)_j, (\mathbf{P}_B)_j, \beta\right) \\ &= (1-\beta)(\mathbf{P}_A)_j + \beta(\mathbf{P}_B)_j.\end{aligned} \tag{12.5}$$

The interpolated pose of the whole skeleton is simply the set of interpolated poses for all of the joints:

$$\mathbf{P}_{\text{LERP}}^{\text{skel}} = \left\{(\mathbf{P}_{\text{LERP}})_j\right\}\Big|_{j=0}^{N-1}. \tag{12.6}$$

In these equations, β is called the *blend percentage* or *blend factor*. When $\beta = 0$, the final pose of the skeleton will exactly match $\mathbf{P}_A^{\text{skel}}$; when $\beta = 1$, the final pose will match $\mathbf{P}_B^{\text{skel}}$. When β is between zero and one, the final pose is an intermediate between the two extremes. This effect is illustrated in Figure 12.11.

We've glossed over one small detail here: We are linearly interpolating *joint poses*, which means interpolating 4×4 *transformation matrices*. But, as we saw in Chapter 5, interpolating matrices directly is not practical. This is one of the reasons why local poses are usually expressed in SRT format—doing so allows us to apply the LERP operation defined in Section 5.2.5 to each component of the SRT individually. The linear interpolation of the translation component $\mathbf{T}$ of an SRT is just a straightforward vector LERP:

$$\begin{aligned}(\mathbf{T}_{\text{LERP}})_j &= \text{LERP}\left((\mathbf{T}_A)_j, (\mathbf{T}_B)_j, \beta\right) \\ &= (1-\beta)(\mathbf{T}_A)_j + \beta(\mathbf{T}_B)_j.\end{aligned} \tag{12.7}$$

The linear interpolation of the rotation component is a quaternion LERP or SLERP (spherical linear interpolation):

$$\begin{aligned}(\mathbf{Q}_{\text{LERP}})_j &= \text{normalize}\left(\text{LERP}\left((\mathbf{Q}_A)_j, (\mathbf{Q}_B)_j, \beta\right)\right) \\ &= \text{normalize}\left((1-\beta)(\mathbf{Q}_A)_j + \beta(\mathbf{Q}_B)_j\right).\end{aligned} \tag{12.8}$$

or

$$\begin{aligned}(\mathbf{Q}_{\text{SLERP}})_j &= \text{SLERP}\left((\mathbf{Q}_A)_j, (\mathbf{Q}_B)_j, \beta\right) \\ &= \frac{\sin\left((1-\beta)\theta\right)}{\sin(\theta)}(\mathbf{Q}_A)_j + \frac{\sin(\beta\theta)}{\sin(\theta)}(\mathbf{Q}_B)_j.\end{aligned} \tag{12.9}$$

Finally, the linear interpolation of the scale component is either a scalar or vector LERP, depending on the type of scale (uniform or nonuniform scale) supported by the engine:

$$\begin{aligned}(\mathbf{S}_{\text{LERP}})_j &= \text{LERP}\left((\mathbf{S}_A)_j, (\mathbf{S}_B)_j, \beta\right) \\ &= (1-\beta)(\mathbf{S}_A)_j + \beta(\mathbf{S}_B)_j.\end{aligned} \tag{12.10}$$

or

$$(S_{\text{LERP}})_j = \text{LERP}\left((S_A)_j, (S_B)_j, \beta\right) = (1-\beta)(S_A)_j + \beta(S_B)_j. \tag{12.11}$$

When linearly interpolating between two skeletal poses, the most natural-looking intermediate pose is generally one in which each joint pose is interpolated independently of the others, in the space of that joint's immediate parent. In other words, pose blending is generally performed on *local poses*. If we were to blend global poses directly in model space, the results would tend to look biomechanically implausible.

Because pose blending is done on local poses, the linear interpolation of any one joint's pose is totally independent of the interpolations of the other joints in the skeleton. This means that linear pose interpolation can be performed entirely in parallel on multiprocessor architectures.

12.6.2 Applications of LERP Blending

Now that we understand the basics of LERP blending, let's have a look at some typical gaming applications.

12.6.2.1 Temporal Interpolation

As we mentioned in Section 12.4.1.1, game animations are almost never sampled exactly on integer frame indices. Because of variable frame rate, the player might actually see frames 0.9, 1.85 and 3.02, rather than frames 1, 2 and 3 as one might expect. In addition, some animation compression techniques involve storing only disparate key frames, spaced at uneven intervals across the clip's local timeline. In either case, we need a mechanism for finding intermediate poses between the sampled poses that are actually present in the animation clip.

LERP blending is typically used to find these intermediate poses. As an example, let's imagine that our animation clip contains evenly spaced pose samples at times 0, Δt, $2\Delta t$, $3\Delta t$ and so on. To find a pose at time $t = 2.18\Delta t$, we simply find the linear interpolation between the poses at times $2\Delta t$ and $3\Delta t$, using a blend percentage of $\beta = 0.18$.

In general, we can find the pose at time t given pose samples at any two times t_1 and t_2 that bracket t, as follows:

$$\mathbf{P}_j(t) = \text{LERP}\left(\mathbf{P}_j(t_1), \mathbf{P}_j(t_2), \beta(t)\right) \tag{12.12}$$

$$= (1-\beta(t))\,\mathbf{P}_j(t_1) + \beta(t)\mathbf{P}_j(t_2), \tag{12.13}$$

where the blend factor $\beta(t)$ can be determined by the ratio

$$\beta(t) = \frac{t - t_1}{t_2 - t_1}. \tag{12.14}$$

12.6.2.2 Motion Continuity: Cross-Fading

Game characters are animated by piecing together a large number of fine-grained animation clips. If your animators are any good, the character will appear to move in a natural and physically plausible way *within* each individual clip. However, it is notoriously difficult to achieve the same level of quality when transitioning from one clip to the next. The vast majority of the "pops" we see in game animations occur when the character transitions from one clip to the next.

Ideally, we would like the movements of each part of a character's body to be perfectly smooth, even during transitions. In other words, the three-dimensional paths traced out by each joint in the skeleton as it moves should contain no sudden "jumps." We call this *C0 continuity*; it is illustrated in Figure 12.29.

Not only should the paths themselves be continuous, but their first derivatives (velocity) should be continuous as well. This is called *C1 continuity* (or continuity of velocity and momentum). The perceived quality and realism of an animated character's movement improves as we move to higher- and higher-order continuity. For example, we might want to achieve *C2 continuity*, in which the second derivatives of the motion paths (acceleration curves) are also continuous.

Strict mathematical continuity up to C1 or higher is often infeasible to achieve. However, LERP-based animation blending can be applied to achieve a reasonably pleasing form of C0 motion continuity. It usually also does a pretty good job of approximating C1 continuity. When applied to transitions between clips in this manner, LERP blending is sometimes called *cross-fading*. LERP blending can introduce unwanted artifacts, such as the dreaded "sliding feet" problem, so it must be applied judiciously.

To cross-fade between two animations, we overlap the timelines of the two clips by some reasonable amount, and then blend the two clips together. The blend percentage β starts at zero at time t_{start}, meaning that we see only clip A

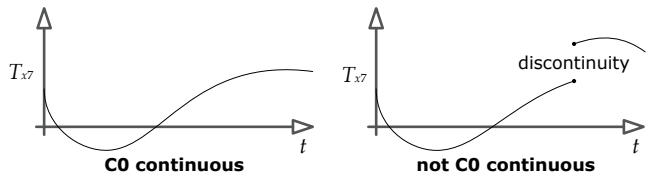

Figure 12.29. The channel function on the left has C0 continuity, while the path on the right does not.

when the cross-fade begins. We gradually increase β until it reaches a value of one at time t_{end}. At this point only clip B will be visible, and we can retire clip A altogether. The time interval over which the cross-fade occurs ($\Delta t_{\text{blend}} = t_{\text{end}} - t_{\text{start}}$) is sometimes called the *blend time*.

Types of Cross-Fades

There are two common ways to perform a cross-blended transition:

- *Smooth transition*. Clips A and B both play simultaneously as β increases from zero to one. For this to work well, the two clips must be looping animations, and their timelines must be synchronized so that the positions of the legs and arms in one clip match up roughly with their positions in the other clip. (If this is not done, the cross-fade will often look totally unnatural.) This technique is illustrated in Figure 12.30.
- *Frozen transition*. The local clock of clip A is stopped at the moment clip B starts playing. Thus, the pose of the skeleton from clip A is frozen while clip B gradually takes over the movement. This kind of transitional blend works well when the two clips are unrelated and cannot be time-synchronized, as they must be when performing a *smooth* transition. This approach is depicted in Figure 12.31.

We can also control how the blend factor β varies during the transition. In Figure 12.30 and Figure 12.31, the blend factor varied linearly with time. To achieve an even smoother transition, we could vary β according to a cubic function of time, such as a one-dimensional Bézier. When such a curve is applied to a currently running clip that is being blended out, it is known as an *ease-out curve*; when it is applied to a new clip that is being blended in, it is known as an *ease-in curve*. This is shown in Figure 12.32.

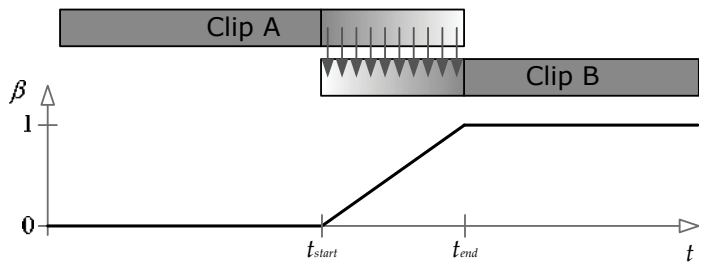

Figure 12.30. A smooth transition, in which the local clocks of both clips keep running during the transition.

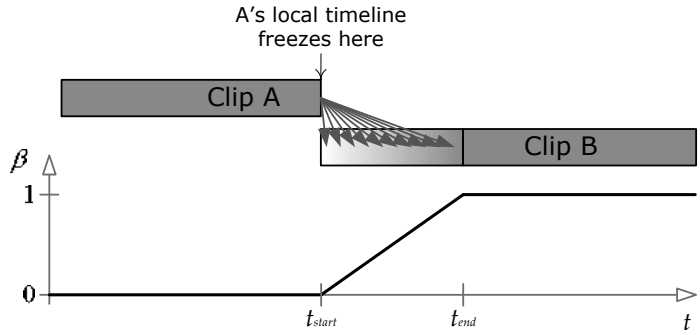

Figure 12.31. A frozen transition, in which clip A's local clock is stopped during the transition.

The equation for a Bézier ease-in/ease-out curve is given below. It returns the value of β at any time t within the blend interval. β_{start} is the blend factor at the start of the blend interval t_{start}, and β_{end} is the final blend factor at time t_{end}. The parameter u is the *normalized time* between t_{start} and t_{end}, and for convenience we'll also define $v = 1 - u$ (the *inverse* normalized time). Note that the Bézier tangents T_{start} and T_{end} are taken to be equal to the corresponding blend factors β_{start} and β_{end}, because this yields a well-behaved curve for our purposes:

$$\begin{aligned}
\text{let } u &= \left(\frac{t - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}}\right) \\
\text{and } v &= 1 - u. \\
\beta(t) &= (v^3)\beta_{\text{start}} + (3v^2u)T_{\text{start}} + (3vu^2)T_{\text{end}} + (u^3)\beta_{\text{end}} \\
&= (v^3 + 3v^2u)\beta_{\text{start}} + (3vu^2 + u^3)\beta_{\text{end}}.
\end{aligned}$$

Core Poses

This is an appropriate time to mention that motion continuity can actually be

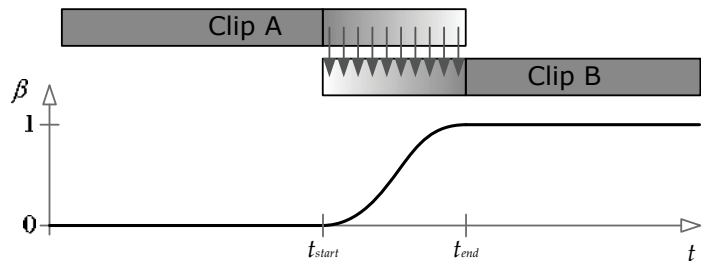

Figure 12.32. A smooth transition, with a cubic ease-in/ease-out curve applied to the blend factor.

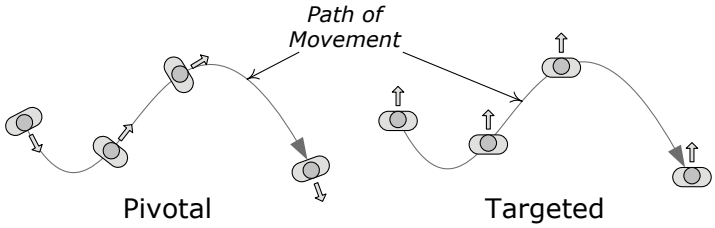

Figure 12.33. In pivotal movement, the character faces the direction she is moving and pivots about her vertical axis to turn. In targeted movement, the movement direction need not match the facing direction.

achieved *without* blending if the animator ensures that the last pose in any given clip matches the first pose of the clip that follows it. In practice, animators often decide upon a set of *core poses*—for example, we might have a core pose for standing upright, one for crouching, one for lying prone and so on. By making sure that the character starts in one of these core poses at the beginning of every clip and returns to a core pose at the end, C0 continuity can be achieved by simply ensuring that the core poses match when animations are spliced together. C1 or higher-order motion continuity can also be achieved by ensuring that the character's movement at the end of one clip smoothly transitions into the motion at the start of the next clip. This can be achieved by authoring a single smooth animation and then breaking it into two or more clips.

12.6.2.3 Directional Locomotion

LERP-based animation blending is often applied to character locomotion. When a real human being walks or runs, he can change the direction in which he is moving in two basic ways: First, he can turn his entire body to change direction, in which case he always faces in the direction he's moving. I'll call this *pivotal movement*, because the person pivots about his vertical axis when he turns. Second, he can keep facing in one direction while walking forward, backward or sideways (known as *strafing* in the gaming world) in order to move in a direction that is independent of his facing direction. I'll call this *targeted movement*, because it is often used in order to keep one's eye—or one's weapon—trained on a target while moving. These two movement styles are illustrated in Figure 12.33.

Targeted Movement

To implement *targeted movement*, the animator authors three separate looping

animation clips—one moving forward, one strafing to the left, and one strafing to the right. I'll call these *directional locomotion clips*. The three directional clips are arranged around the circumference of a semicircle, with forward at 0 degrees, left at 90 degrees and right at -90 degrees. With the character's facing direction fixed at 0 degrees, we find the desired movement direction on the semicircle, select the two adjacent movement animations and blend them together via LERP-based blending. The blend percentage β is determined by how close the angle of movement is to the angles of two adjacent clips. This is illustrated in Figure 12.34.

Note that we did not include backward movement in our blend, for a full circular blend. This is because blending between a sideways strafe and a backward run cannot be made to look natural in general. The problem is that when strafing to the left, the character usually crosses its right foot in front of its left so that the blend into the pure forward run animation looks correct. Likewise, the right strafe is usually authored with the left foot crossing in front of the right. When we try to blend such strafe animations directly into a backward run, one leg will start to pass through the other, which looks extremely awkward and unnatural. There are a number of ways to solve this problem. One feasible approach is to define two hemispherical blends, one for forward motion and one for backward motion, each with strafe animations that have been crafted to work properly when blended with the corresponding straight run. When passing from one hemisphere to the other, we can play some kind of explicit transition animation so that the character has a chance to adjust its gait and leg crossing appropriately.

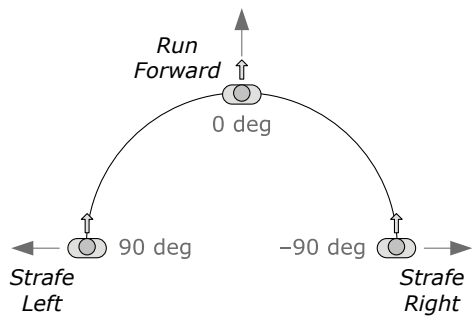

Figure 12.34. Targeted movement can be implemented by blending together looping locomotion clips that move in each of the four principal directions.

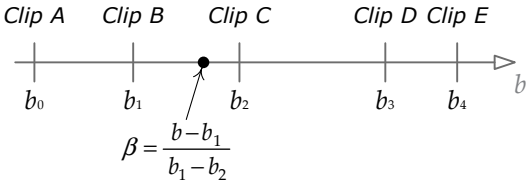

Figure 12.35. A generalized linear blend between N animation clips.

Pivotal Movement

To implement *pivotal movement*, we can simply play the forward locomotion loop while rotating the entire character about its vertical axis to make it turn. Pivotal movement looks more natural if the character's body doesn't remain bolt upright when it is turning—real humans tend to lean into their turns a little bit. We could try slightly tilting the vertical axis of the character as a whole, but that would cause problems with the inner foot sinking into the ground while the outer foot comes off the ground. A more natural-looking result can be achieved by animating three variations on the basic forward walk or run—one going perfectly straight, one making an extreme left turn and one making an extreme right turn. We can then LERP-blend between the straight clip and the extreme left turn clip to implement any desired lean angle.

12.6.3 Complex LERP Blends

In a real game engine, characters make use of a wide range of complex blends for various purposes. It can be convenient to "prepackage" certain commonly used types of complex blends for ease of use. In the following sections, we'll investigate a few popular types of prepackaged complex blends.

12.6.3.1 Generalized One-Dimensional LERP Blending

LERP blending can be easily extended to more than two animation clips, using a technique I call *one-dimensional LERP blending*. We define a new blend parameter b that lies in any linear range desired (e.g., from -1 to $+1$, or from 0 to 1, or even from 27 to 136). Any number of clips can be positioned at arbitrary points along this range, as shown in Figure 12.35. For any given value of b, we select the two clips immediately adjacent to it and blend them together using Equation (12.5). If the two adjacent clips lie at points b_1 and b_2, then the blend percentage β can be determined using a technique analogous to that used in Equation (12.14), as follows:

$$\beta(t) = \frac{b - b_1}{b_2 - b_1}. \tag{12.15}$$

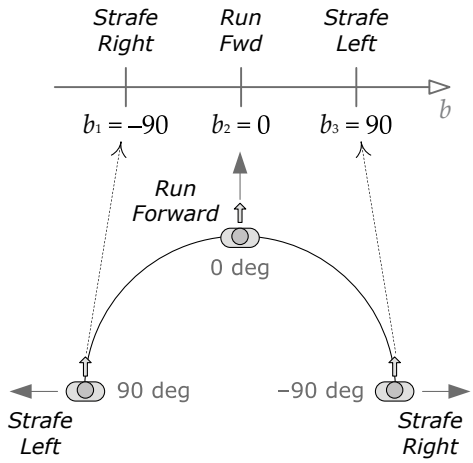

Figure 12.36. The directional clips used in targeted movement can be thought of as a special case of one-dimensional LERP blending.

Targeted movement is just a special case of one-dimensional LERP blending. We simply straighten out the circle on which the directional animation clips were placed and use the movement direction angle θ as the parameter b (with a range of -90 to 90 degrees). Any number of animation clips can be placed onto this blend range at arbitrary angles. This is shown in Figure 12.36.

12.6.3.2 Simple Two-Dimensional LERP Blending

Sometimes we would like to smoothly vary *two* aspects of a character's motion simultaneously. For example, we might want the character to be capable of aiming his weapon vertically and horizontally. Or we might want to allow our character to vary her pace length and the separation of her feet as she moves. We can extend one-dimensional LERP blending to two dimensions in order to achieve these kinds of effects.

If we know that our 2D blend involves only four animation clips, and if those clips are positioned at the four corners of a square region, then we can find a blended pose by performing two 1D blends. Our generalized blend factor b becomes a two-dimensional blend vector $\mathbf{b} = \begin{bmatrix} b_x & b_y \end{bmatrix}$. If $\mathbf{b}$ lies within the square region bounded by our four clips, we can find the resulting pose by following these steps:

1. Using the horizontal blend factor b_x, find two intermediate poses, one between the top two animation clips and one between the bottom two clips. These two poses can be found by performing two simple one-

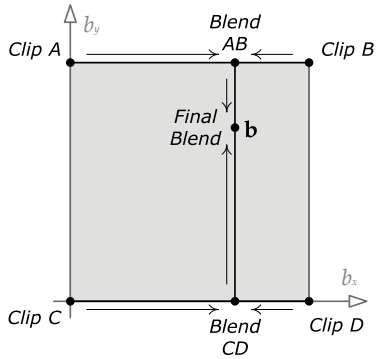

Figure 12.37. A simple formulation for 2D animation blending between four clips at the corners of a square region.

dimensional LERP blends.

2. Using the vertical blend factor b_y, find the final pose by LERP-blending the two intermediate poses together.

This technique is illustrated in Figure 12.37.

12.6.3.3 Triangular Two-Dimensional LERP Blending

The simple 2D blending technique we investigated in the previous section only works when the animation clips we wish to blend lie at the corners of a rectangular region. How can we blend between an arbitrary number of clips positioned at arbitrary locations in our 2D blend space?

Let's imagine that we have three animation clips that we wish to blend together. Each clip, designated by the index i, corresponds to a particular blend coordinate $\mathbf{b}_i = \begin{bmatrix} b_{ix} & b_{iy} \end{bmatrix}$ in our two-dimensional blend space; these three blend coordinates form a triangle within the blend space. Each of the three clips defines a set of joint poses $\left\{ (\mathbf{P}_i)_j \right\}\Big|_{j=0}^{N-1}$, where $(\mathbf{P}_i)_j$ is the pose of joint j as defined by clip i, and N is the number of joints in the skeleton. We wish to find the interpolated pose of the skeleton corresponding to an arbitrary point $\mathbf{b}$ within the triangle, as illustrated in Figure 12.38.

But how can we calculate a LERP blend between three animation clips? Thankfully, the answer is simple: the LERP function can actually operate on any number of inputs, because it is really just a *weighted average*. As with any weighted average, the weights must add to one. In the case of a two-input LERP blend, we used the weights β and $(1-\beta)$, which of course add to one. For a three-input LERP, we simply use three weights, α, β and $\gamma = (1-\alpha-\beta)$.

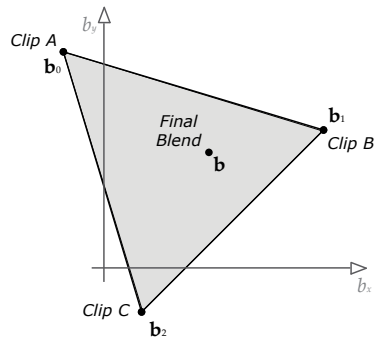

Figure 12.38. Two-dimensional animation blending between three animation clips.

Then we calculate the LERP as follows:

$$(\mathbf{P}_{\text{LERP}})_j = \alpha\,(\mathbf{P}_0)_j + \beta\,(\mathbf{P}_1)_j + \gamma\,(\mathbf{P}_2)_j\,. \tag{12.16}$$

Given the two-dimensional blend vector $\mathbf{b}$, we find the blend weights α, β and γ by finding the *barycentric coordinates* of the point $\mathbf{b}$ relative to the triangle formed by the three clips in two-dimensional blend space (http://en.wikipedia.org/wiki/Barycentric_coordinates_%28mathematics%29). In general, the barycentric coordinates of a point $\mathbf{b}$ within a triangle with vertices $\mathbf{b}_1$, $\mathbf{b}_2$ and $\mathbf{b}_3$ are three scalar values (α, β, γ) that satisfy the relations

$$\mathbf{b} = \alpha\mathbf{b}_0 + \beta\mathbf{b}_1 + \gamma\mathbf{b}_2, \tag{12.17}$$

and

$$\alpha + \beta + \gamma = 1.$$

These are exactly the weights we seek for our three-clip weighted average. Barycentric coordinates are illustrated in Figure 12.39.

Note that plugging the barycentric coordinate $(1, 0, 0)$ into Equation (12.17) yields $\mathbf{b}_0$, while $(0, 1, 0)$ gives us $\mathbf{b}_1$ and $(0, 0, 1)$ produces $\mathbf{b}_2$. Likewise, plugging these blend weights into Equation (12.16) gives us poses $(\mathbf{P}_0)_j$, $(\mathbf{P}_1)_j$ and $(\mathbf{P}_2)_j$ for each joint j, respectively. Furthermore, the barycentric coordinate $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ lies at the centroid of the triangle and gives us an *equal* blend between the three poses. This is exactly what we'd expect.

12.6.3.4 Generalized Two-Dimensional LERP Blending

The barycentric coordinate technique can be extended to an arbitrary number of animation clips positioned at arbitrary locations within the two-dimensional blend space. We won't describe it in its entirety here, but the basic idea is to use a technique known as *Delaunay triangulation* (http://en.wikipedia.org/

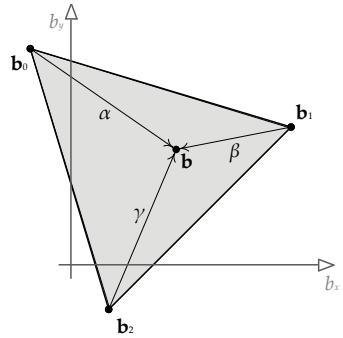

Figure 12.39. Various barycentric coordinates within a triangle.

wiki/Delaunay_triangulation) to find a set of triangles given the positions of the various animation clips $\mathbf{b}_i$. Once the triangles have been determined, we can find the triangle that encloses the desired point $\mathbf{b}$ and then perform a three-clip LERP blend as described above. This technique was used in FIFA soccer by EA Sports in Vancouver, implemented within their proprietary "ANT" animation framework. It is shown in Figure 12.40.

12.6.4 Partial-Skeleton Blending

A human being can control different parts of his or her body independently. For example, I can wave my right arm while walking and pointing at something with my left arm. One way to implement this kind of movement in a

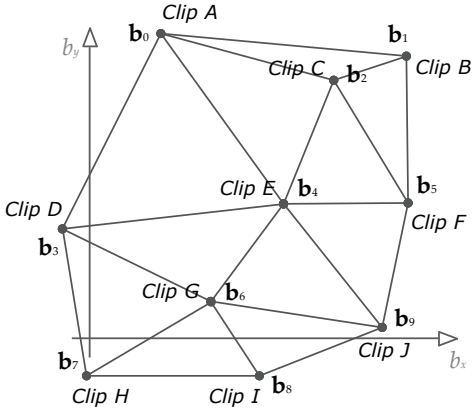

Figure 12.40. Delaunay triangulation between an arbitrary number of animation clips positioned at arbitrary locations in two-dimensional blend space.

game is via a technique known as *partial-skeleton blending*.

Recall from Equations (12.5) and (12.6) that when doing regular LERP blending, the same blend percentage β was used for every joint in the skeleton. Partial-skeleton blending extends this idea by permitting the blend percentage to vary on a per-joint basis. In other words, for each joint j, we define a separate blend percentage β_j. The set of all blend percentages for the entire skeleton $\{\beta_j\}|_{j=0}^{N-1}$ is sometimes called a *blend mask* because it can be used to "mask out" certain joints by setting their blend percentages to zero.

As an example, let's say we want our character to wave at someone using his right arm and hand. Moreover, we want him to be able to wave whether he's walking, running or standing still. To implement this using partial blending, the animator defines three full-body animations: *Walk*, *Run* and *Stand*. The animator also creates a single waving animation, *Wave*. A blend mask is created in which the blend percentages are zero everywhere except for the right shoulder, elbow, wrist and finger joints, where they are equal to one:

$$\beta_j = \begin{cases} 1 & \text{when } j \text{ within right arm,} \\ 0 & \text{otherwise.} \end{cases}$$

When *Walk*, *Run* or *Stand* is LERP-blended with *Wave* using this blend mask, the result is a character who appears to be walking, running or standing while waving his right arm.

Partial blending is useful, but it has a tendency to make a character's movements look unnatural. This occurs for two basic reasons:

- An abrupt change in the per-joint blend factors can cause the movements of one part of the body to appear disconnected from the rest of the body. In our example, the blend factors change abruptly at the right shoulder joint. Hence the animation of the upper spine, neck and head are being driven by one animation, while the right shoulder and arm joints are being entirely driven by a different animation. This can look odd. The problem can be mitigated somewhat by gradually changing the blend factors rather than doing it abruptly. (In our example, we might select a blend percentage of 0.9 at the right shoulder, 0.5 on the upper spine and 0.2 on the neck and mid-spine.)
- The movements of a real human body are never totally independent. For example, one would expect a person's wave to look more "bouncy" and out of control when he or she is running than when he or she is standing still. Yet with partial blending, the right arm's animation will be identical no matter what the rest of the body is doing. This problem is difficult to

overcome using partial blending. Instead, many game developers have turned to a more natural-looking technique known as *additive blending*.

12.6.5 Additive Blending

Additive blending approaches the problem of combining animations in a totally new way. It introduces a new kind of animation called a *difference clip*, which, as its name implies, represents the difference between two regular animation clips. A difference clip can be added onto a regular animation clip in order to produce interesting variations in the pose and movement of the character. In essence, a difference clip encodes the *changes* that need to be made to one pose in order to transform it into another pose. Difference clips are often called *additive animation clips* in the game industry. We'll stick with the term *difference clip* in this book because it more accurately describes what is going on.

Consider two input clips called the *source clip* (S) and the *reference clip* (R). Conceptually, the difference clip is $D = S - R$. If a difference clip D is added to its original reference clip, we get back the source clip $(S = D + R)$. We can also generate animations that are partway between R and S by adding a percentage of D to R, in much the same way that LERP blending finds intermediate animations between two extremes. However, the real beauty of the additive blending technique is that once a difference clip has been created, it can be added to other unrelated clips, not just to the original reference clip. We'll call these animations *target clips* and denote them with the symbol T.

As an example, if the reference clip has the character running normally and the source clip has him running in a tired manner, then the difference clip will contain only the changes necessary to make the character look "tired" while running. If this difference clip is now applied to a clip of the character walking, the resulting animation can make the character look tired while walking. A whole host of interesting and very natural-looking animations can be created by adding a single difference clip onto various "regular" animation clips, or a collection of difference clips can be created, each of which produces a different effect when added to a single target animation.

12.6.5.1 Mathematical Formulation

A difference animation D is defined as the difference between some source animation S and some reference animation R. So conceptually, the difference pose (at a single point in time) is $D = S - R$. Of course, we're dealing with joint poses, not scalar quantities, so we cannot simply subtract the poses. In general, a joint pose is a 4×4 affine transformation matrix that transforms

points and vectors from the child joint's local space to the space of its parent joint. The matrix equivalent of subtraction is multiplication by the inverse matrix. So given the source pose $\mathbf{S}_j$ and the reference pose $\mathbf{R}_j$ for any joint j in the skeleton, we can define the difference pose $\mathbf{D}_j$ at that joint as follows. (For this discussion, we'll drop the C $\rightarrow$ P or $j \rightarrow p(j)$ subscript, as it is understood that we are dealing with child-to-parent pose matrices.)

$$\mathbf{D}_j = \mathbf{S}_j \mathbf{R}_j^{-1}.$$

"Adding" a difference pose $\mathbf{D}_j$ onto a target pose $\mathbf{T}_j$ yields a new additive pose $\mathbf{A}_j$. This is achieved by simply concatenating the difference transform and the target transform as follows:

$$\mathbf{A}_j = \mathbf{D}_j \mathbf{T}_j = \left(\mathbf{S}_j \mathbf{R}_j^{-1} \right) \mathbf{T}_j. \tag{12.18}$$

We can verify that this is correct by looking at what happens when the difference pose is "added" back onto the original reference pose:

$$\begin{aligned} \mathbf{A}_j &= \mathbf{D}_j \mathbf{R}_j \\ &= \mathbf{S}_j \mathbf{R}_j^{-1} \mathbf{R}_j \\ &= \mathbf{S}_j. \end{aligned}$$

In other words, adding the difference animation D back onto the original reference animation R yields the source animation S, as we'd expect.

Temporal Interpolation of Difference Clips

As we learned in Section 12.4.1.1, game animations are almost never sampled on integer frame indices. To find a pose at an arbitrary time t, we must often *temporally interpolate* between adjacent pose samples at times t_1 and t_2. Thankfully, difference clips can be temporally interpolated just like their non-additive counterparts. We can simply apply Equations (12.12) and (12.14) directly to our difference clips as if they were ordinary animations.

Note that a difference animation can only be found when the input clips S and R are of the same duration. Otherwise there would be a period of time during which either S or R is undefined, meaning D would be undefined as well.

Additive Blend Percentage

In games, we often wish to blend in only a percentage of a difference animation to achieve varying degrees of the effect it produces. For example, if a difference clip causes the character to turn his head 80 degrees to the right, blending in

50% of the difference clip should make him turn his head only 40 degrees to the right.

To accomplish this, we turn once again to our old friend LERP. We wish to interpolate between the unaltered target animation and the new animation that would result from a full application of the difference animation. To do this, we extend Equation (12.18) as follows:

$$\begin{aligned}\mathbf{A}_j &= \text{LERP}\left(\mathbf{T}_j, \mathbf{D}_j\mathbf{T}_j, \beta\right) \\ &= (1-\beta)\left(\mathbf{T}_j\right) + \beta\left(\mathbf{D}_j\mathbf{T}_j\right).\end{aligned} \tag{12.19}$$

As we saw in Chapter 5, we cannot LERP matrices directly. So Equation (11.16) must be broken down into three separate interpolations for S, Q and T, just as we did in Equations (12.7) through (12.11).

12.6.5.2 Additive Blending versus Partial Blending

Additive blending is similar in some ways to partial blending. For example, we can take the difference between a standing clip and a clip of standing while waving the right arm. The result will be almost the same as using a partial blend to make the right arm wave. However, additive blends suffer less from the "disconnected" look of animations combined via partial blending. This is because, with an additive blend, we are not replacing the animation for a subset of joints or interpolating between two potentially unrelated poses. Rather, we are adding movement to the original animation—possibly across the entire skeleton. In effect, a difference animation "knows" how to change a character's pose in order to get him to do something specific, like being tired, aiming his head in a certain direction, or waving his arm. These changes can be applied to a reasonably wide variety of animations, and the result often looks very natural.

12.6.5.3 Limitations of Additive Blending

Of course, additive animation is not a silver bullet. Because it adds movement to an existing animation, it can have a tendency to over-rotate the joints in the skeleton, especially when multiple difference clips are applied simultaneously. As a simple example, imagine a target animation in which the character's left arm is bent at a 90 degree angle. If we add a difference animation that also rotates the elbow by 90 degrees, then the net effect would be to rotate the arm by $90 + 90 = 180$ degrees. This would cause the lower arm to interpenetrate the upper arm—not a comfortable position for most individuals!

Clearly we must be careful when selecting the reference clip and also when choosing the target clips to which to apply it. Here are some simple rules of thumb:

- Keep hip rotations to a minimum in the reference clip.
- The shoulder and elbow joints should usually be in neutral poses in the reference clip to minimize over-rotation of the arms when the difference clip is added to other targets.
- Animators should create a new difference animation for each core pose (e.g., standing upright, crouched down, lying prone, etc.). This allows the animator to account for the way in which a real human would move when in each of these stances.

These rules of thumb can be a helpful starting point, but the only way to really learn how to create and apply difference clips is by trial and error or by apprenticing with animators or engineers who have experience creating and applying difference animations. If your team hasn't used additive blending in the past, expect to spend a significant amount of time learning the art of additive blending.

12.6.6 Applications of Additive Blending

12.6.6.1 Stance Variation

One particularly striking application of additive blending is *stance variation*. For each desired stance, the animator creates a one-frame difference animation. When one of these single-frame clips is additively blended with a base animation, it causes the entire stance of the character to change drastically while he continues to perform the fundamental action he's supposed to perform. This idea is illustrated in Figure 12.41.

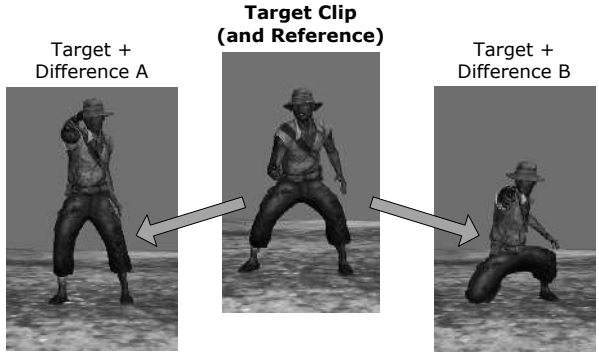

Figure 12.41. Two single-frame difference animations A and B can cause a target animation clip to assume two totally different stances. (Character from *Uncharted: Drake's Fortune*, © 2007/® SIE. Created and developed by Naughty Dog.)

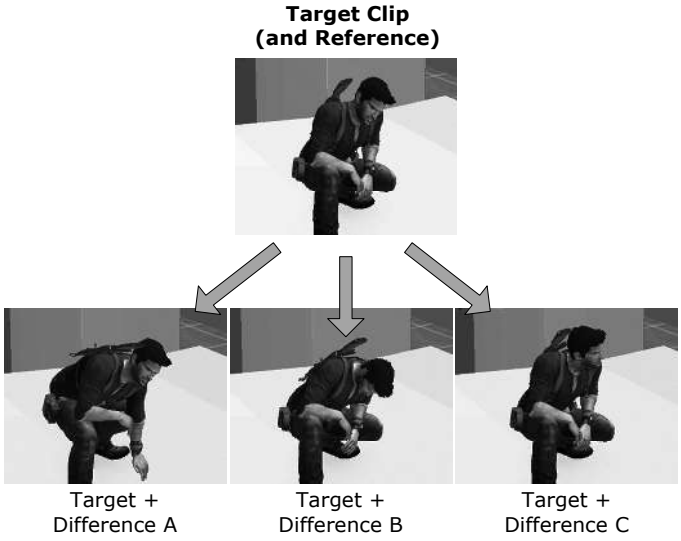

Figure 12.42. Additive blends can be used to add variation to a repetitive idle animation. Images courtesy of Naughty Dog, Inc., © 2014/™ SIE.

12.6.6.2 Locomotion Noise

Real humans don't run exactly the same way with every footfall—there is variation in their movement over time. This is especially true if the person is distracted (for example, by attacking enemies). Additive blending can be used to layer randomness, or reactions to distractions, on top of an otherwise entirely repetitive locomotion cycle. This is illustrated in Figure 12.42.

12.6.6.3 Aim and Look-At

Another common use for additive blending is to permit the character to look around or to aim his weapon. To accomplish this, the character is first animated doing some action, such as running, with his head or weapon facing straight ahead. Then the animator changes the direction of the head or the aim of the weapon to the extreme right and saves off a one-frame or multi-frame difference animation. This process is repeated for the extreme left, up and down directions. These four difference animations can then be additively blended onto the original straight-ahead animation clip, causing the character to aim right, left, up, down or anywhere in between.

The angle of the aim is governed by the additive blend factor of each clip. For example, blending in 100% of the right additive causes the character to aim as far right as possible. Blending 50% of the left additive causes him to aim at

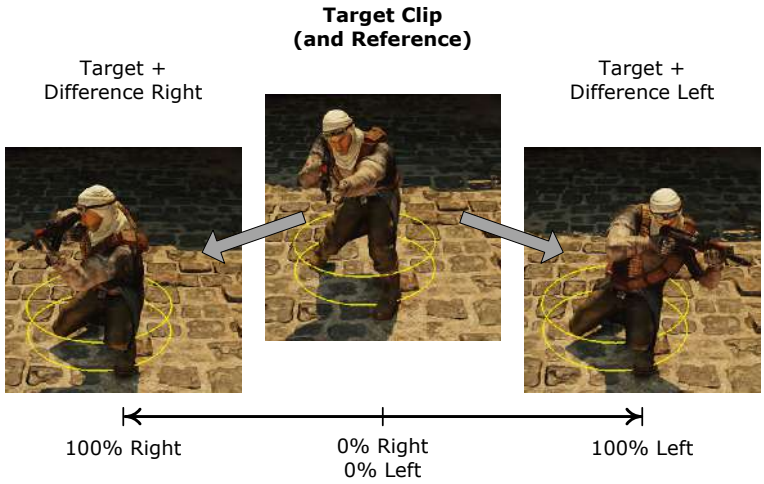

Figure 12.43. Additive blending can be used to aim a weapon. Screenshots courtesy of Naughty Dog, Inc., © 2014/™ SIE.

an angle that is one-half of his leftmost aim. We can also combine this with an up or down additive to aim diagonally. This is demonstrated in Figure 12.43.

12.6.6.4 Overloading the Time Axis

It's interesting to note that the time axis of an animation clip needn't be used to represent time. For example, a three-frame animation clip could be used to provide three aim poses to the engine—a left aim pose on frame 1, a forward aim pose on frame 2 and a right aim pose on frame 3. To make the character aim to the right, we can simply fix the local clock of the aim animation on frame 3. To perform a 50% blend between aiming forward and aiming right, we can dial in frame 2.5. This is a great example of leveraging existing features of the engine for new purposes.

12.7 Post-Processing

Once a skeleton has been posed by one or more animation clips and the results have been blended together using linear interpolation or additive blending, it is often necessary to modify the pose prior to rendering the character. This is called *animation post-processing*. In this section, we'll look at a few of the most common kinds of animation post-processing.

12.7.1 Procedural Animations

A *procedural animation* is any animation generated at runtime rather than being driven by data exported from an animation tool such as Maya. Sometimes, hand-animated clips are used to pose the skeleton initially, and then the pose is modified in some way via procedural animation as a post-processing step. A procedural animation can also be used as an input to the system in place of a hand-animated clip.

For example, imagine that a regular animation clip is used to make a vehicle appear to be bouncing up and down on the terrain as it moves. The direction in which the vehicle travels is under player control. We would like to adjust the rotation of the front wheels and steering wheel so that they move convincingly when the vehicle is turning. This can be done by post-processing the pose generated by the animation. Let's assume that the original animation has the front tires pointing straight ahead and the steering wheel in a neutral position. We can use the current angle of turn to create a quaternion about the vertical axis that will deflect the front tires by the desired amount. This quaternion can be multiplied with the front tire joints' Q channel to produce the final pose of the tires. Likewise, we can generate a quaternion about the axis of the steering column and multiply it into the steering wheel joint's Q channel to deflect it. These adjustments are made to the *local pose*, prior to global pose calculation and matrix palette generation (see Section 12.5).

As another example, let's say that we wish to make the trees and bushes in our game world sway naturally in the wind and get brushed aside when characters move through them. We can do this by modeling the trees and bushes as skinned meshes with simple skeletons. Procedural animation can be used, in place of or in addition to hand-animated clips, to cause the joints to move in a natural-looking way. We might apply one or more sinusoids, or a Perlin noise function, to the rotation of various joints to make them sway in the breeze, and when a character moves through a region containing a bush or grass, we can deflect its root joint quaternion radially outward to make it appear to be pushed over by the character.

12.7.2 Inverse Kinematics

Let's say we have an animation clip in which a character leans over to pick up an object from the ground. In Maya, the clip looks great, but in our production game level, the ground is not perfectly flat, so sometimes the character's hand misses the object or appears to pass through it. In this case, we would like to adjust the final pose of the skeleton so that the hand lines up exactly with the target object. A technique known as *inverse kinematics* (IK) can be used to make

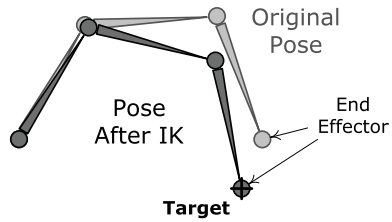

Figure 12.44. Inverse kinematics attempts to bring an end effector joint into a target global pose by minimizing the error between them.

this happen.

A regular animation clip is an example of *forward kinematics* (FK). In forward kinematics, the input is a set of local joint poses, and the output is a global pose and a skinning matrix for each joint. Inverse kinematics goes in the other direction: The input is the desired global pose of a single joint, which is known as the *end effector*. We solve for the *local* poses of other joints in the skeleton that will bring the end effector to the desired location.

Mathematically, IK boils down to an *error minimization* problem. As with most minimization problems, there might be one solution, many or none at all. This makes intuitive sense: If I try to reach a doorknob that is on the other side of the room, I won't be able to reach it without walking over to it. IK works best when the skeleton starts out in a pose that is reasonably close to the desired target. This helps the algorithm to focus in on the "closest" solution and to do so in a reasonable amount of processing time. Figure 12.44 shows IK in action.

Imagine a two-joint skeleton, each of which can rotate only about a single axis. The rotation of these two joints can be described by a two-dimensional angle vector $\theta = \begin{bmatrix}\theta_1 & \theta_2\end{bmatrix}$. The set of all possible angles for our two joints forms a two-dimensional space called *configuration space*. Obviously, for more complex skeletons with more degrees of freedom per joint, configuration space becomes multidimensional, but the concepts described here work equally well no matter how many dimensions we have.

Now imagine plotting a three-dimensional graph, where for each combination of joint rotations (i.e., for each point in our two-dimensional configuration space), we plot the distance from the end effector to the desired target. An example of this kind of plot is shown in Figure 12.45. The "valleys" in this three-dimensional surface represent regions in which the end effector is as close as possible to the target. When the height of the surface is zero, the end effector has reached its target. Inverse kinematics, then, attempts to find

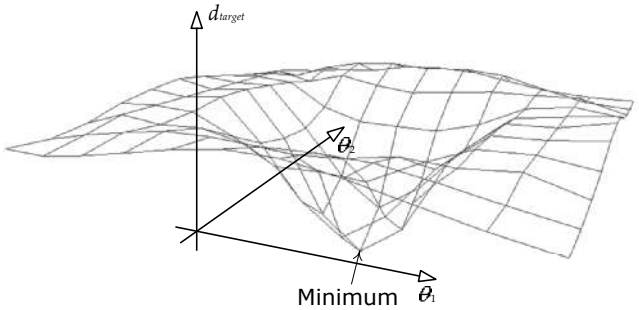

Figure 12.45. A three-dimensional plot of the distance from the end effector to the target for each point in two-dimensional configuration space. IK finds the local minimum.

minima (low points) on this surface.

We won't get into the details of solving the IK minimization problem here. You can read more about IK at http://en.wikipedia.org/wiki/Inverse_kinematics and in Jason Weber's article, "Constrained Inverse Kinematics" [47].

12.7.3 Rag Dolls

A character's body goes limp when he dies or becomes unconscious. In such situations, we want the body to react in a physically realistic way with its surroundings. To do this, we can use a *rag doll*. A rag doll is a collection of physically simulated rigid bodies, each one representing a semi-rigid part of the character's body, such as his lower arm or his upper leg. The rigid bodies are constrained to one another at the joints of the character in such a way as to produce natural-looking "lifeless" body movement. The positions and orientations of the rigid bodies are determined by the physics system and are then used to drive the positions and orientations of certain key joints in the character's skeleton. The transfer of data from the physics system to the skeleton is typically done as a post-processing step.

To really understand rag doll physics, we must first have an understanding of how the collision and physics systems work. Rag dolls are covered in more detail in Sections 13.4.8.7 and 13.5.3.8.

12.8 Compression Techniques

Animation data can take up a lot of memory. A single joint pose might be composed of ten floating-point channels (three for translation, four for rotation and